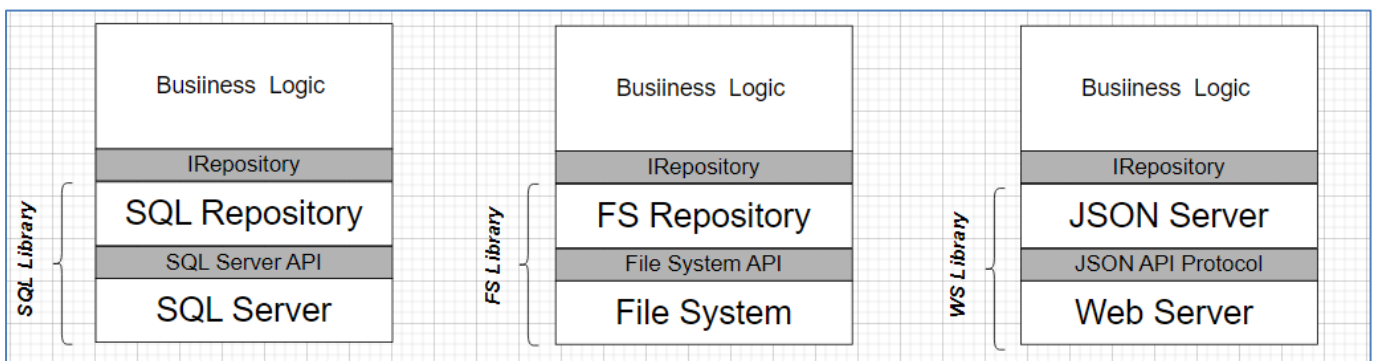
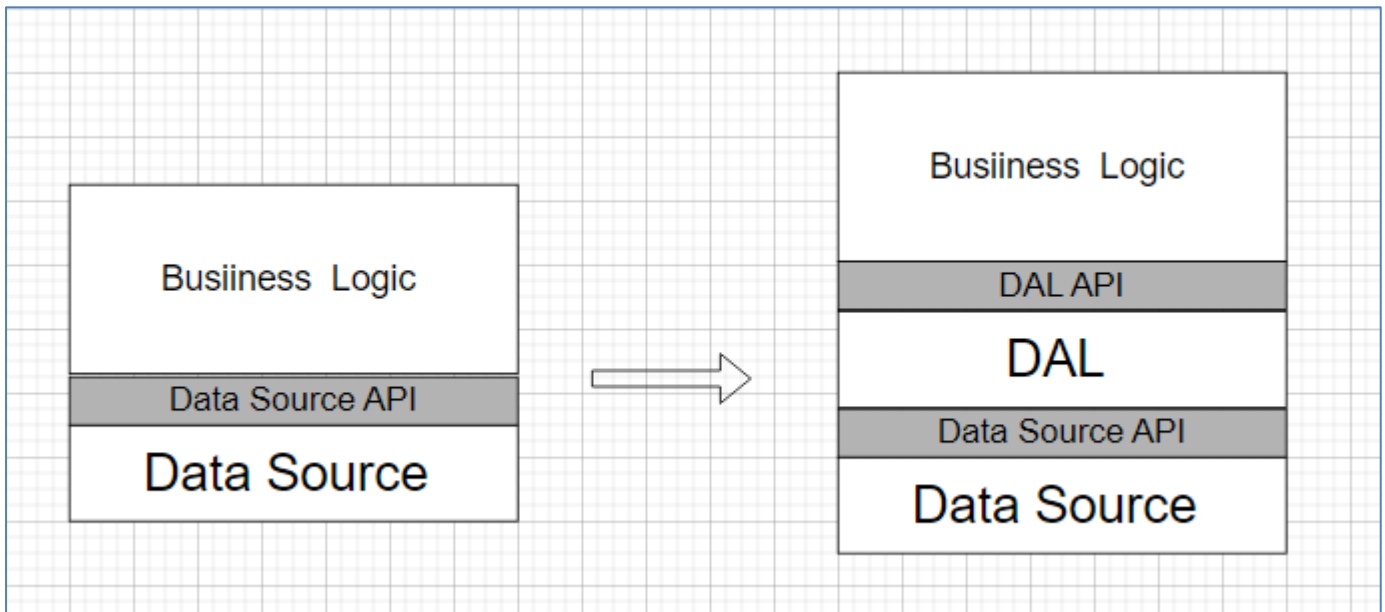
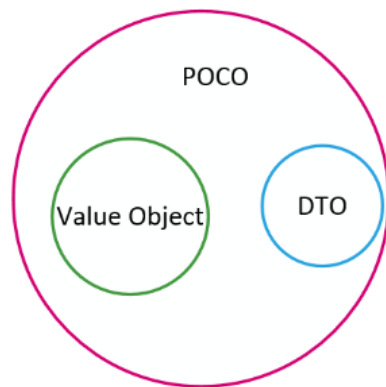


Паттерны DAO, Repository

1. **DAL: Data Access Layer** – программный слой в архитектуре приложения, предназначенный для работы с источниками данных. Разделение ответственности, абстракция, независимость от источника данных. Реализация: паттерны DAO (Data Access Object), Repository, ORM (Object-Relation Mapping).



2. **DTO (Data Transfer Object)**: класс, содержащий данные без логики для работы с ними (структуры); применяется для передачи данных между различными приложениями или между слоями одного и того же приложения; рассматривается как хранилище данных, единственная цель которого – передать эти данные.
3. **Value Object**: объект DDD-домена; подчиняется тем же правилам, что и Entity, но у него нет идентичности – два Value-объекта с одинаковыми свойствами не различаются.
4. **POCO (Plain Old CLR Object)**: аналогия POJO (Java), могут содержать логику, может быть и Value и Entity.



- DTO != Value Object
- DTO $\subset$ POCO
- Value Object $\subset$ POCO

	DTO	Value Object	POCO
Contains Data	yes	yes	yes
Contains Logic	no	yes	yes/no
Has Own Identity	no	no	yes/no

5. DTO: пример

```
public class dtoCelebrity // DTO
{
    public string FirstName { get; set; } = null!; // имя
    public string Surname { get; set; } = null!; // фамилия
}
```

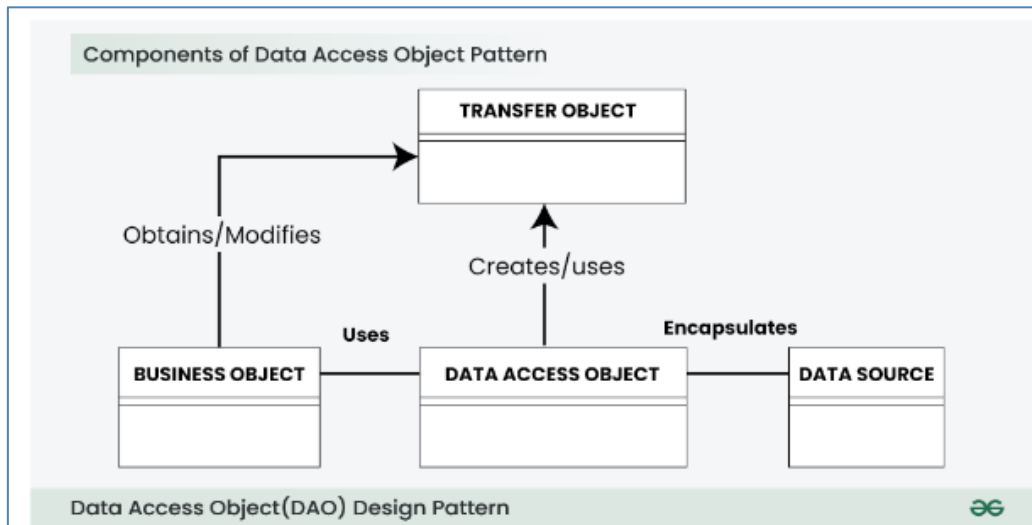
6. Value Object: пример

```
public class valCelebrity // Value
{
    public valCelebrity(string FirstName, string Surname)
    {
        this.FirstName = FirstName;
        this.Surname = Surname;
    }
    public string FirstName { get; set; } = " "; // имя
    public string Surname { get; set; } = " "; // фамилия
    public string getShort { get { return $"{this.FirstName.Substring(0, 1)}.{this.Surname}"; } }
}
```

7. Entity: пример

```
public class Celebrity // Entity
{
    public Celebrity(int id, string FirstName, string Surname)
    {
        this.id = id;
        this.FirstName = FirstName;
        this.Surname = Surname;
    }
    public int id;
    public string FirstName { get; set; } = " "; // имя
    public string Surname { get; set; } = " "; // фамилия
    public string getShort { get { return $"{this.FirstName.Substring(0, 1)}.{this.Surname}"; } }
}
```

8. **DAO:** Data Object Access; паттерн проектирования; абстрактный интерфейс к данным; класс, содержащий CRUD (Create, Read, Update, Delete) для конкретной сущности; отделяет бизнес-логику от механизмов сохранения; четко не описан.



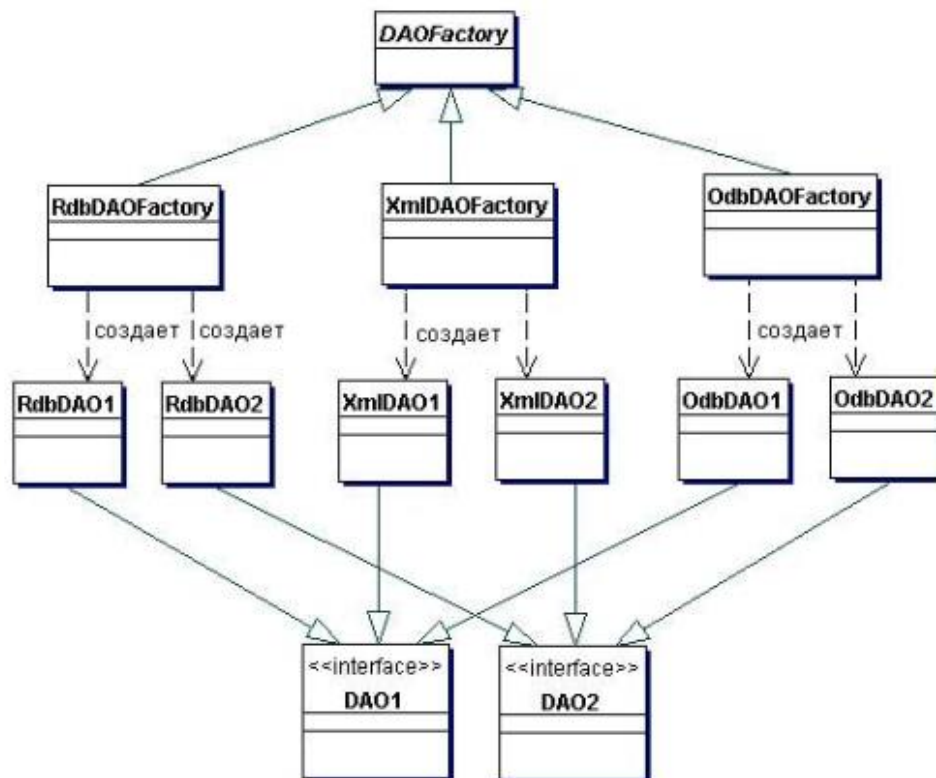
9. **DAO: Business Object** использует DAO-объект для работы с данными; DAO-Объект создает Transfer Object для объекты работы с данными; Data Source – источник данных для подключения.

```

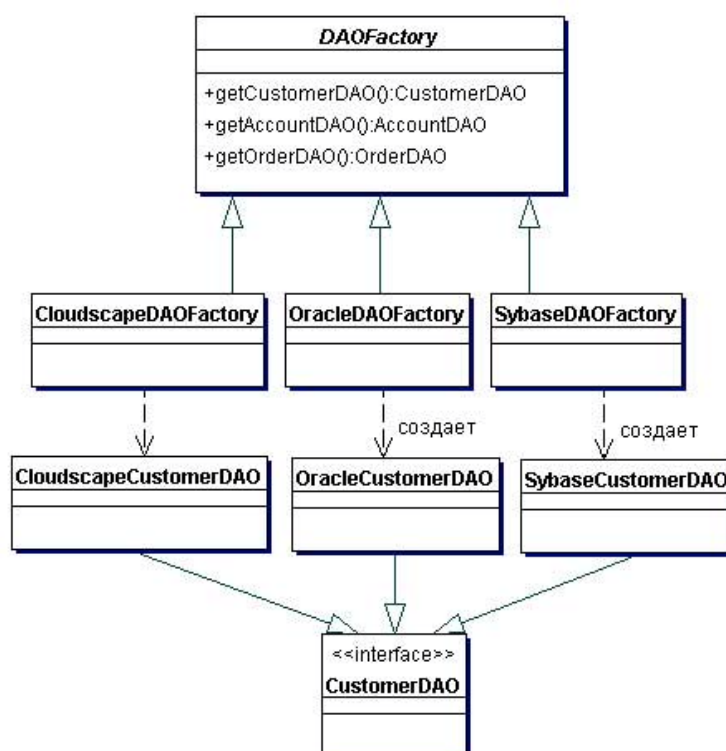
public class DAOCelebrities
{
    public class dtoCelebrity
    {
        public string FirstName { get; set; } = " "; // имя
        public string Surname { get; set; } = " "; // фамилия
    }

    public dtoCelebrity Get(int id) ...
    public int Create (dtoCelebrity celebrity) ...
    public bool Update(int id, Celebrity celebrity) ...
    public bool Delete(int id) ...
}
  
```

10. **DAO/Abstract Factory:** DAOFactory – абстрактный класс/интерфейс; RdbDAOFactory, XmlDAOFactory, OdbDAOFacto

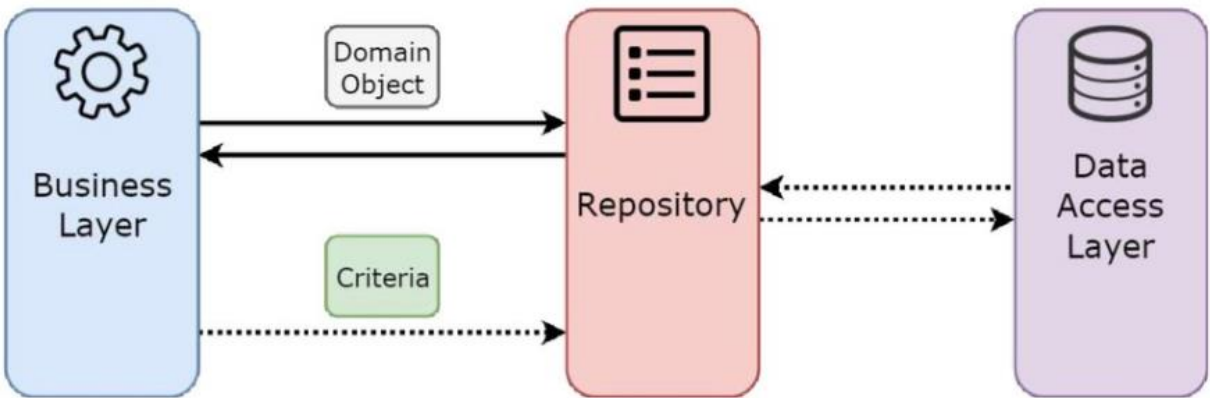


11. **Repository**: паттерн проектирования, применяемый при работе с данными; предназначен для разделения бизнес-логики и от слоя доступа к данным. Два подхода DDD: Database Drive Design (разработка начинается с разработки БД), Domain Drive Design (разработка начинается с проектирования бизнес-логики)



(домена), Эрик Эванс). Repository ассоциируется с Domain Drive Design.

- 12. **Repository**: посредник между слоем доступа к данным и доменным слоем; in-memory коллекция доменных объектов.
- 13. **Repository**: клиенты создают декларативные описания запросов и передают их в репозиторий для выполнения. Мартин Фаулер.



- 14. **Repository**: интерфейс

```
[
{
  "Id": 1,
  "Firstname": "Noam",
  "Surname": "Chomsky",
  "PhotoPath": "/Photo/Chomsky.jpg"
},
{
  "Id": 2,
  "Firstname": "Tim",
  "Surname": "Berners-Lee",
  "PhotoPath": "/Photo/Berners-Lee.jpg"
},
{
  "Id": 3,
  "Firstname": "Edgar",
  "Surname": "Codd",
  "PhotoPath": "/Photo/Codd.jpg"
},
{
  "Id": 4,
  "Firstname": "Donald",
  "Surname": "Knuth",
  "PhotoPath": "/Photo/Knuth.jpg"
},
{
  "Id": 5,
  "Firstname": "Linus",
  "Surname": "Torvalds",
  "PhotoPath": "/Photo/Torvalds.jpg"
},
{
  "Id": 6,
  "Firstname": "John",
  "Surname": "Neumann",
  "PhotoPath": "/Photo/Neumann.jpg"
},
{
  "Id": 7,
  "Firstname": "Edsger",
  "Surname": "Dijkstra",
  "PhotoPath": "/Photo/Dijkstra.jpg"
},
{
  "Id": 8,
  "Firstname": "Marvin",
  "Surname": "Minsky",
  "PhotoPath": "/Photo/Minsky.jpg"
}
]
```

```
public interface ICelebrity<T> : IDisposable
{
    List<T> GetAllCelebrities(); // получить все Знаменитости
    T? GetCelebrityById(int Id); // получить Знаменитость по Id
    bool DelCelebrity(int id); // удалить Знаменитость по Id
    bool AddCelebrity(T celebrity); // добавить Знаменитость
    int AddCelebrityAndGetId(T celebrity); // добавить Знаменитость и получить его Id
    bool UpdCelebrity(int id, T celebrity); // изменить Знаменитость по Id
    int GetCelebrityIdByName(string name); // получить первый Id по вхождению подстроки
    int SaveChanges(); // сохранить изменения = количество изменений
}
```

15. Repository: реализация JSON-Repository

```
public record Celebrity( // Entity
    int Id,              // идентификатор
    string Firstname,    // имя
    string Surname,      // фамилия
    string PhotoPath     // путь к файлу фотографии
);

public class Repository : ICelebrity<Celebrity> // реализация Repository
{
    public static string JSONFileName = "Celebrities.json"; // имя JSON-файла
    public string BasePath { get; private set; }           // базовый путь
    public string FullBasePath { get => Path.Combine(this.BasePath, JSONFileName); } // полный путь к JSON-файлу
    List<Celebrity> celebrities = null!;                   // список знаменитостей
    int NChanges = 0;                                     // количество изменений

    private Repository(string basepath) {...}
    public static ICelebrity<Celebrity> Create(string basepath) { return new Repository(basepath); }

    public bool AddCelebrity(Celebrity celebrity) {...}
    public int AddCelebrityAndGetId(Celebrity celebrity) {...}
    public bool DelCelebrity(int id) {...}
    public void Dispose() {...}
    public List<Celebrity> GetAllCelebrities() {...}
    public Celebrity? GetCelebrityById(int id) {...}
    public int GetCelebrityIdByName(string name) {...}
    public bool UpdCelebrity(int id, Celebrity celebrity) {...}
    public int SaveChanges() {...}
}
```

```
private Repository(string basepath)
{
    this.celebrities = new();
    this.BasePath = Path.Combine(Directory.GetCurrentDirectory(), basepath);
    if (File.Exists(this.FullBasePath))
    {
        FileStream fs = new FileStream(this.FullBasePath, FileMode.Open, FileAccess.Read);
        fs.Seek(0, SeekOrigin.Begin);
        List<Celebrity>? c = JsonSerializer.DeserializeAsync<List<Celebrity>>(fs).Result; // JSON --> List<Celebrity>
        if (c != null) this.celebrities = c;
        fs.Close();
    }
}
```

```
public List<Celebrity> GetAllCelebrities()
{
    return this.celebrities;
}
```

```
public bool AddCelebrity(Celebrity celebrity)
{
    int id = this.celebrities.Count > 0 ? this.celebrities.Max<Celebrity>(c => c.Id)+1:1;
    this.celebrities.Add(new Celebrity(id,celebrity.Firstname,celebrity.Surname,celebrity.PhotoPath));
    this.NChanges++;
    return true;
}
```

```
public int AddCelebrityAndGetId(Celebrity celebrity)
{
    int id = this.celebrities.Count > 0 ? this.celebrities.Max<Celebrity>(c => c.Id) + 1 : 1;
    this.celebrities.Add(new Celebrity(id, celebrity.Firstname, celebrity.Surname, celebrity.PhotoPath));
    this.NChanges++;
    return id;
}
```

```

public bool UpdCelebrity(int id, Celebrity celebrity)
{
    int ntx = this.celebrities.FindIndex(c => c.Id == id);
    if (ntx >= 0)
    {
        this.celebrities[ntx] = new Celebrity(id, celebrity.Firstname, celebrity.Surname, celebrity.PhotoPath);
        this.NChanges++;
    }
    return (ntx >= 0);
}

```

```

public bool DelCelebrity(int id)
{
    int ntx = this.celebrities.FindIndex(c => c.Id == id);
    if (ntx >= 0)
    {
        this.celebrities.RemoveAt(ntx); this.NChanges++;
    }

    return (ntx >= 0);
}

```

```

public Celebrity? GetCelebrityById(int id)
{
    return this.celebrities.FirstOrDefault(c => c.Id == id);
}

```

```

public int GetCelebrityIdByName(string name)
{
    Celebrity? celebrity = this.celebrities.FirstOrDefault(c => (c.Firstname + "|" + c.Surname).Contains(name));
    return celebrity != null ? celebrity.Id : -1;
}

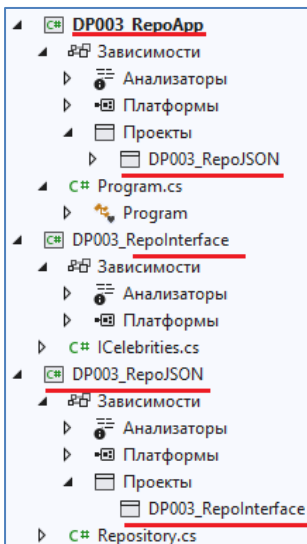
```

```

public int SaveChanges()
{
    int rc = this.NChanges;
    if (rc > 0)
    {
        FileStream fs = new FileStream(this.FullBasePath, FileMode.Truncate, FileAccess.Write);
        fs.Seek(0, SeekOrigin.Begin);
        Task t = JsonSerializer.SerializeAsync<List<Celebrity>>(fs, this.celebrities, new JsonSerializerOptions { WriteIndented = true });
        t.Wait();
        fs.Close();
        this.NChanges = 0;
    }
    return rc;
}

```

16. Repository: структура проекта



17. Repository: тест

```
private static void Main(string[] args)
{
    Repository.JSONFileName = "Celebrities.json";
    using (ICelebrity<Celebrity> repo = Repository.Create("Celebrities")) // директорий (в текущем директории приложения)
    {
        Console.WriteLine("----- GetAllCelebrities -----");
        {
            List<Celebrity> celebrities = repo.GetAllCelebrities();
            celebrities.ForEach(c => Console.WriteLine($"Id = {c.Id}, Firstname = {c.Firstname}, Surname = {c.Surname}, PhotoPath = {c.PhotoPath} "));
        }
        Console.WriteLine("----- AddCelebrity -----");
        {
            repo.AddCelebrity(new Celebrity(0, "Marlin", "Minsky", "/Photo/Minsky.jpg"));
            List<Celebrity> celebrities = repo.GetAllCelebrities();
            celebrities.ForEach(c => Console.WriteLine($"Id = {c.Id}, Firstname = {c.Firstname}, Surname = {c.Surname}, PhotoPath = {c.PhotoPath} "));
        }
        Console.WriteLine("----- GetCelebrityIdByName, UpdCelebrity -----");
        {
            int id = repo.GetCelebrityIdByName("Marlin");
            if (id >= 0) repo.UpdCelebrity(id, new Celebrity(0, "Marvin", "Minsky", "/Photo/Minsky.jpg"));
            List<Celebrity> celebrities = repo.GetAllCelebrities();
            celebrities.ForEach(c => Console.WriteLine($"Id = {c.Id}, Firstname = {c.Firstname}, Surname = {c.Surname}, PhotoPath = {c.PhotoPath} "));
        }
        Console.WriteLine("----- GetCelebrityIdByName, DelCelebrity -----");
        {
            int id = repo.GetCelebrityIdByName("Minsky");
            if (id >= 0) repo.DelCelebrity(id);
            List<Celebrity> celebrities = repo.GetAllCelebrities();
            celebrities.ForEach(c => Console.WriteLine($"Id = {c.Id}, Firstname = {c.Firstname}, Surname = {c.Surname}, PhotoPath = {c.PhotoPath} "));
        }
        Console.WriteLine("----- AddCelebrityAndGetId -----");
        {
            int id = repo.AddCelebrityAndGetId(new Celebrity(0, "Marvin", "Minsky", "/Photo/Minsky.jpg"));
            Console.WriteLine("Minsky Id = {0}", id);
            List<Celebrity> celebrities = repo.GetAllCelebrities();
            celebrities.ForEach(c => Console.WriteLine($"Id = {c.Id}, Firstname = {c.Firstname}, Surname = {c.Surname}, PhotoPath = {c.PhotoPath} "));
        }
        {
            Console.WriteLine("SaveChanges = {0}", repo.SaveChanges());
        }
    }
    using (ICelebrity<Celebrity> repo = Repository.Create("Celebrities"))
    {
        List<Celebrity> celebrities = repo.GetAllCelebrities();
        celebrities.ForEach(c => Console.WriteLine($"Id = {c.Id}, Firstname = {c.Firstname}, Surname = {c.Surname}, PhotoPath = {c.PhotoPath} "));
    }
}
```

Консоль отладки Microsoft Visual Studio

```
----- GetAllCelebrities -----
Id = 1, Firstname = Noam, Surname = Chomsky, PhotoPath = /Photo/Chomsky.jpg
Id = 2, Firstname = Tim, Surname = Berners-Lee, PhotoPath = /Photo/Berners-Lee.jpg
Id = 3, Firstname = Edgar, Surname = Codd, PhotoPath = /Photo/Codd.jpg
Id = 4, Firstname = Donald, Surname = Knuth, PhotoPath = /Photo/Knuth.jpg
Id = 5, Firstname = Linus, Surname = Torvalds, PhotoPath = /Photo/Torvalds.jpg
Id = 6, Firstname = John, Surname = Neumann, PhotoPath = /Photo/Neumann.jpg
Id = 7, Firstname = Edsger, Surname = Dijkstra, PhotoPath = /Photo/Dijkstra.jpg
Id = 8, Firstname = Marvin, Surname = Minsky, PhotoPath = /Photo/Minsky.jpg
----- AddCelebrity -----
Id = 1, Firstname = Noam, Surname = Chomsky, PhotoPath = /Photo/Chomsky.jpg
Id = 2, Firstname = Tim, Surname = Berners-Lee, PhotoPath = /Photo/Berners-Lee.jpg
Id = 3, Firstname = Edgar, Surname = Codd, PhotoPath = /Photo/Codd.jpg
Id = 4, Firstname = Donald, Surname = Knuth, PhotoPath = /Photo/Knuth.jpg
Id = 5, Firstname = Linus, Surname = Torvalds, PhotoPath = /Photo/Torvalds.jpg
Id = 6, Firstname = John, Surname = Neumann, PhotoPath = /Photo/Neumann.jpg
Id = 7, Firstname = Edsger, Surname = Dijkstra, PhotoPath = /Photo/Dijkstra.jpg
Id = 8, Firstname = Marvin, Surname = Minsky, PhotoPath = /Photo/Minsky.jpg
Id = 9, Firstname = Marlin, Surname = Minsky, PhotoPath = /Photo/Minsky.jpg
----- GetCelebrityIdByName, UpdCelebrity -----
Id = 1, Firstname = Noam, Surname = Chomsky, PhotoPath = /Photo/Chomsky.jpg
Id = 2, Firstname = Tim, Surname = Berners-Lee, PhotoPath = /Photo/Berners-Lee.jpg
Id = 3, Firstname = Edgar, Surname = Codd, PhotoPath = /Photo/Codd.jpg
Id = 4, Firstname = Donald, Surname = Knuth, PhotoPath = /Photo/Knuth.jpg
Id = 5, Firstname = Linus, Surname = Torvalds, PhotoPath = /Photo/Torvalds.jpg
Id = 6, Firstname = John, Surname = Neumann, PhotoPath = /Photo/Neumann.jpg
Id = 7, Firstname = Edsger, Surname = Dijkstra, PhotoPath = /Photo/Dijkstra.jpg
Id = 8, Firstname = Marvin, Surname = Minsky, PhotoPath = /Photo/Minsky.jpg
Id = 9, Firstname = Marvin, Surname = Minsky, PhotoPath = /Photo/Minsky.jpg
----- GetCelebrityIdByName, DelCelebrity -----
Id = 1, Firstname = Noam, Surname = Chomsky, PhotoPath = /Photo/Chomsky.jpg
Id = 2, Firstname = Tim, Surname = Berners-Lee, PhotoPath = /Photo/Berners-Lee.jpg
Id = 3, Firstname = Edgar, Surname = Codd, PhotoPath = /Photo/Codd.jpg
Id = 4, Firstname = Donald, Surname = Knuth, PhotoPath = /Photo/Knuth.jpg
Id = 5, Firstname = Linus, Surname = Torvalds, PhotoPath = /Photo/Torvalds.jpg
Id = 6, Firstname = John, Surname = Neumann, PhotoPath = /Photo/Neumann.jpg
Id = 7, Firstname = Edsger, Surname = Dijkstra, PhotoPath = /Photo/Dijkstra.jpg
Id = 9, Firstname = Marvin, Surname = Minsky, PhotoPath = /Photo/Minsky.jpg
----- AddCelebrityAndGetId -----
```

18. КОНЕЦ